

实验一：分治算法

1. 实验目的

- 1、掌握分治算法的设计思想与方法，
- 2、熟练使用高级编程语言实现分治算法，
- 3、通过对比简单算法以及不同的分治求解思想，理解算法复杂度。

2. 实验学时

4 学时。

3. 实验问题

求解凸包问题：输入是平面上 n 个点的集合 Q ，凸包问题是要输出一个 Q 的凸包。其中， Q 的凸包是一个凸多边形 P ， Q 中的点或者在 P 上或者在 P 中。（详情请见课件）

4. 实验步骤

4.1 实现基于枚举方法的凸包求解算法

提示：考虑 Q 中的任意四个点 A 、 B 、 C 、 D ，如果 A 处于 BCD 构成的三角形内部，那么 A 一定不属于凸包 P 的顶点集合。

4.2 实现基于 **Graham-Scan** 的凸包求解算法

提示：具体算法思想见课件。

4.3 实现基于分治思想的凸包求解算法

提示：具体算法思想见课件。

4.4 对比三种凸包求解算法

- (1) 实现随机生成正方形 $(0,0)-(0,100)-(100,100)-(100,0)$ 内的点集合 Q 的算法；
- (2) 利用点集合生成算法自动生成大小不同数据集合，如点数大小分别为 $(1000, 2000, 3000\dots)$ 的数据集合；

- (3) 对每个算法，针对不同大小的数据集合，运行算法并记录算法运行时间；
- (4) 对每个算法，绘制算法性能曲线，对比算法。

实验二：搜索算法

1. 实验目的

- 1、掌握搜索算法的基本设计思想与方法，
- 2、掌握 A*算法的设计思想与方法，
- 3、熟练使用高级编程语言实现搜索算法，
- 4、利用实验测试给出的搜索算法的正确性。

2. 实验学时

4 学时。

3. 实验问题

寻路问题。以图 1 为例，输入一个方格表示的地图，要求用 A*算法找到并输出从起点（在方格中标示字母 S）到终点（在方格中标示字母 T）的代价最小的路径。有如下条件及要求：

- (1) 每一步都落在方格中，而不是横竖线的交叉点。
- (2) 灰色格子表示障碍，无法通行。
- (3) 在每个格子处，若无障碍，下一步可以达到八个相邻的格子，并且只可以到达无障碍的相邻格子。其中，向上、下、左、右四个方向移动的代价为 1，向四个斜角方向移动的代价为 $\sqrt{2}$ 。
- (4) 在一些特殊格子上行走要花费额外的地形代价。比如，黄色格子代表沙漠，经过它的代价为 4；蓝色格子代表溪流，经过它的代价为 2；白色格子为普通地形，经过它的代价为 0。
- (5) 经过一条路径总的代价为移动代价+地形代价。其中移动代价是路径上所做的所有移动的代价的总和；地形代价为路径上除起点外所有格子的地形代价的总和。比如，在下图的示例中，路径 A→B→C 的代价为 $\sqrt{2}+1(\text{移动})+0(\text{地形})$ ，而路径 D→E→F 的代价为 $2(\text{移动})+6(\text{地形})$ 。

A		
	B	C
D	E	F

4. 实验步骤

4.1 实现单向与双向的 A*搜索算法

以图 1 为样例，实现单向和双向 A*算法并测试输出。输出的路径应该与图 1 中以圆点标示的路径一致，或者略有不同但是有一样的代价。规则和条件如第 3 节所述。

双向 A*算法应当从 S 点和 T 点同时行 A*搜索。

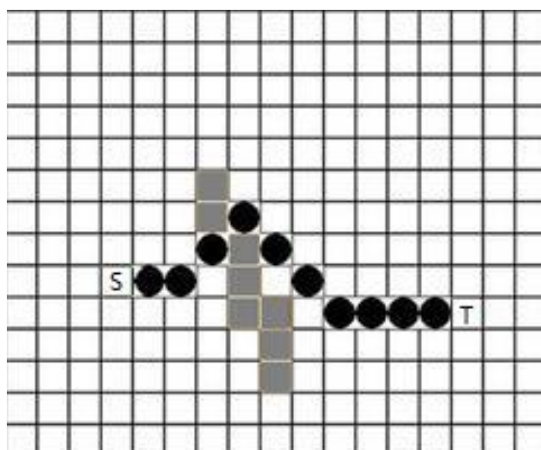


图 1

4.2 测试 A*算法的应用

精灵王子 Haposola 一大早从城堡中起床，要去沙漠中河流消失的地方寻找失落的宝藏。长路漫漫，处处险恶，精灵王子要尽快到达目标地点。

以图 2 为输入，采用单向和双向的 A*算法，寻找地图上给出的起点和终点间的最小代价路径。规则和条件如第 3 节所述。

4.3 结果可视化

以适当的方式将地图可视化呈现，应该体现出方格、起点终点、障碍、地形等地图要素，并将算法计算出的最小代价路径在地图上体现出来。可以不完全遵守样例的形式（如颜色和以圆点表示的路径等），但以上要素应该完整地体现出来。

对于双向 A*算法，应当展示出双向搜索的路径（如从 S 点的搜索结果和从 T 点的搜索结果以不同的颜色表示）。

5. 考核须知

- (1) 需要展示 A*算法在两个输入下（图 1 和图 2）的输出。
- (2) 会根据同学们使用的可视化方法适当增减分数。

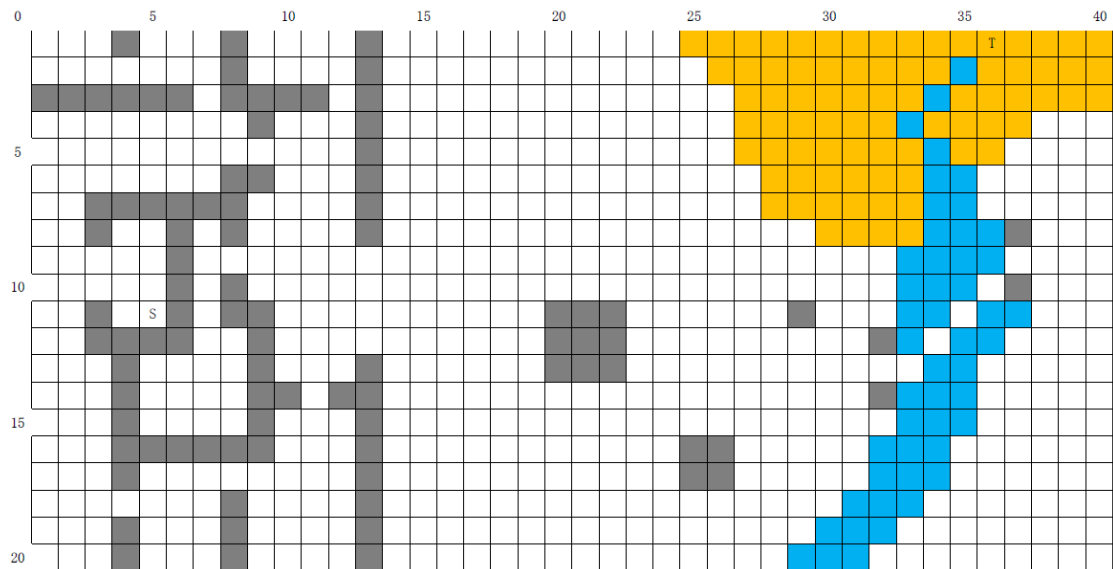


图 2

实验三：近似算法

1. 实验目的

- 1、掌握近似算法的基本设计思想与方法，
- 2、掌握集合覆盖问题近似算法的设计思想与方法，
- 3、熟练使用高级编程语言实现近似算法，
- 4、利用实验测试给出不同近似算法的性能以理解其优缺点。

2. 实验学时

4 学时。

3. 实验问题

集合覆盖问题：

- 输入：有限集 X , X 的子集合族 F , $X = \bigcup_{S \in F} S$
- 输出： $C \subseteq F$, 满足

(1) $X = \bigcup_{S \in \mathcal{C}} S$

(2) $\mathcal{C}$ 是满足条件(1)的最小集族, 即 $|\mathcal{C}|$ 最小.

4. 实验步骤

4.1 实现基于贪心策略的近似算法。

提示：具体算法思想见课件。

4.2 实现一个基于线性规划近似算法。

提示：具体算法思想见课件。要求使用舍入法或随机舍入法。求解线性规划的步骤建议使用现成的库或工具，比如 C++ 的 GBLK 包，Python 的 pulp 包等。

4.3 测试算法性能

实验测试在 $|X|=|F|=100、1000、5000$ 情况，注意打印出可行解方案。

注意：在问题规模为 5000 时算法很有可能运行非常长的时间。

数据生成建议：

(1) 确保生成一组可行解集合

S_0 ：随机选 X 中的 20 个点放入其中， S_1 ：产生一个 1-20 的随机数 n 代表 S_1 集合大小，再随机一个 1- n 的随机数 x 代表需要重 $X-S_0$ 随机选 x 个点放入 S_1 中，从 S_0 中随机选 $n-x$ 个点放入 S_1 ；依次类推，对于 S_{i+1} ，需要从 $X-\bigcup_{j=0}^i S_j$ 中随机选取 x 个点放入 S_{i+1} 中，从 $\bigcup_{j=0}^i S_j$ 随机选 $n-x$ 个点放入 S_{i+1} 中；当 $X-\bigcup_{j=0}^i S_j$ 小于 20 的时候，直接作为最后一个集合。

(2) 生成其余集合

假设上述一共生成集合 y 个，自己再设计随机生成 $|F|-y$ 个集合 S 。

5. 考核须知

(1) 注意实现的时候理解贪心近似算法设计思路。

(2) 需要了解线性规划近似算法的设计思路，并通过实践理解其优缺点。

实验四：快速排序

1 实验目的

- 1、掌握快速排序随机算法的设计思想与方法。
- 2、熟练使用高级编程语言实现不同的快速排序算法。
- 3、利用实验测试给出不同快速排序算法的性能以理解其优缺点。

2 实验学时

4 学时。

3 实验问题

快速排序是算法导论中的经典算法。在本实验中，给定一个长为 n 的整数数组，要求将数组升序排序。

4 实验步骤

4.1 按照算法导论中给出的伪代码实现快速排序

请按照如下伪代码实现算法，并在小样例上测试，保证算法的正确性。

```
QuickSort (A, p, r)
    if p < r
        q = Rand_Partition (A, p, r)
        QuickSort (A, p, q-1)
        QuickSort (A, q+1, r)
Rand_Partition (A, p, r)
    i = Random (p, r)
    exchange A[r] with A[i]
    x = A[r]
    i = p - 1
    for j = p to r - 1
        if A[j] <= x
            i = i + 1
            exchange A[i] with A[j]
    exchange A[i+1] with A[r]
    return i + 1
```

4.2 测试算法在不同输入下的表现

生成 11 个大小为 10^6 的整数数据集，第 i 个子集中存在元素重复 $10^6 \times 10 \times i\%$, $i = 0, 1, 2, \dots, 10$ ，在各个实验数据集上运行算法观察实验现象

并记录实验结果。

4.3 思考并解决问题

调用编程语言库函数中的快速排序算法在各个数据实验数据集上运行算法，观察实验现象并记录实验结果。解释对比结果，改进你实验的算法并将其实现。

实验五：计算 k 位数的随机算法

1. 实验目的

- 1、掌握分治算法和随机算法的设计思想与方法，
- 2、熟练使用高级编程语言实现分治算法和随机算法，
- 3、通过对比不同参数，理解算法具体原理，理解分治算法和随机算法。

2. 实验学时

4 学时。

3. 实验问题

输入：乱序实数数组 $A = \{x_1, x_2, \dots, x_n\}$

输出： A 的第 k 小元素（详情请见课件分治算法、随机算法）

4. 实验步骤

4.1 实现基于分治方法的 k 位数算法

提示：参考第三章 3.5 节。

4.2 实现基于随机方法的 k 位数算法

提示：参考第七章 7.3 节。

4.3 对比两种 k 位数算法

(1) 随机生成大小不同数据集合，如大小分别为(1000, 2000, 5000, 10000, 20000, 50000, 100000)的数据集合；

(2) 对每个算法，针对不同大小的数据集合，运行算法 1000 次并记录算法运行的平均时间；

(3) 将得到的运行时间绘制成曲线，分析并比较两种不同的算法。

4.4 随机算法参数分析

(1) 改变随机 k 位数算法的采样比例 (原始是 $3/4$), 分别为 $(1/8, 1/4, 3/8, \dots, 7/8, 1)$;

注: $|P| \leq 4n^{3/4} + 1$ 此处的 $3/4$ 亦需要修改。

(2) 点数规模是 50000, 运行算法 1000 次, 记录第一次运行算法可以得到 k 位数的次数和运行一次算法的平均时间;

(3) 将一次运行得到正确结果的概率和运行时间绘制成曲线, 分析并比较。

帮助

(1) 记录算法运行时间方法(要求单位精确到毫秒): 利用各种高级语言提供的记录系统当前时间的函数, 在程序中调用算法前记录系统当前时间, 在程序中调用算法后记录系统时间, 利用两个时间的差作为算法运行时间。

(2) 绘制算法性能曲线, 可以利用 Excel 软件的“插入折线图”等类似功能。算法性能曲线应该反映当数据集合从小变大时, 具体算法的运行时间。